

Regex Engines & Command Modes

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>grep 'pattern' file</code>	Basic Regular Expressions (BRE) — metacharacters <code>+</code> , <code>?</code> , <code>(</code> , <code>)</code> , <code>{</code> , <code>}</code> , <code> </code> must be escaped with backslashes (<code>\+</code> , <code>\?</code> , <code>\(</code> , <code>\)</code>) EXAMPLE: <code>grep '\([0-9]\+\)' data.txt</code>
<code>grep -E 'pattern' (or egrep)</code>	Extended Regular Expressions (ERE) — metacharacters <code>+</code> , <code>?</code> , <code>(</code> , <code>)</code> , <code>{</code> , <code>}</code> , <code> </code> are special by default without backslashes EXAMPLE: <code>grep -E '^[0-9]+(\.[0-9]+)?\$' numbers.txt</code>
<code>grep -P 'pattern'</code>	Perl-Compatible Regular Expressions (PCRE) — enables <code>\d</code> , <code>\s</code> , <code>\w</code> , <code>\K</code> , lookaheads (<code>?=...</code>), lookbehinds (<code>?<=...</code>) EXAMPLE: <code>grep -P '\b(?<=\\\$\\s?)\\d+\\b' receipts.txt</code>
<code>grep -F 'string' (or fgrep)</code>	Fixed Strings — searches for exact literal strings without any regex interpretation (fastest search mode) EXAMPLE: <code>grep -F 'user[1].id' config.json</code>

Output & Matching Flags

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>-i / --ignore-case</code>	Case-insensitive search — ignores case distinctions in both pattern and input data EXAMPLE: <code>grep -i 'error' /var/log/syslog</code>
<code>-v / --invert-match</code>	Inverts match — selects lines that do NOT match the given regular expression pattern EXAMPLE: <code>grep -v '^#' config.yml</code> (filters out comments)
<code>-n / --line-number</code>	Prefixes each line of output with its 1-based line number within its input file EXAMPLE: <code>grep -n 'TODO' src/index.ts</code>
<code>-c / --count</code>	Suppresses normal output; prints only the count of matching lines for each input file EXAMPLE: <code>grep -c 'POST /login' access.log</code>
<code>-o / --only-matching</code>	Prints only the matched (non-empty) parts of a matching line, with each part on a separate output line EXAMPLE: <code>grep -oE '[a-zA-Z0-9._%+~]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}' text.txt</code>
<code>-w / --word-regexp</code>	Matches only whole words — pattern must match a full word surrounded by word boundary characters EXAMPLE: <code>grep -w 'cat' animal.txt</code> (matches 'cat', not 'catch')
<code>-x / --line-regexp</code>	Matches whole line only — pattern must match the entire line from start to end (implicit <code>^...\$</code>) EXAMPLE: <code>grep -x 'ENABLED' flags.txt</code>
<code>-m NUM / --max-count=NUM</code>	Stops reading file after finding NUM matching lines (optimizes search on huge log files) EXAMPLE: <code>grep -m 5 'FATAL' application.log</code>
<code>-q / --quiet / --silent</code>	Quiet mode — prints nothing; exits immediately with status 0 if match found, 1 if not (ideal for if-checks) EXAMPLE: <code>if grep -q 'success' status.log; then ... fi</code>

Directories, Files, & Context Control

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>-r / -R / --recursive</code>	Recursively searches all files under current directory; -R follows symbolic links, -r does not EXAMPLE: <code>grep -rn 'API_KEY' .</code>
<code>--include=GLOB</code>	Search only files whose base name matches the specified wildcard GLOB pattern EXAMPLE: <code>grep -rn --include='*.ts' 'interface' src/</code>
<code>--exclude=GLOB / --exclude-dir=DIR</code>	Skip files or directories matching GLOB (e.g. ignore build artifacts or vendor folders) EXAMPLE: <code>grep -rn --exclude-dir={node_modules,.git,dist} 'process' .</code>
<code>-l / --files-with-matches</code>	Suppresses normal output; prints only the names of files containing at least one match EXAMPLE: <code>grep -rl 'deprecated' ./lib/</code>
<code>-L / --files-without-match</code>	Suppresses normal output; prints only the names of files that do NOT contain any match EXAMPLE: <code>grep -rL 'Copyright 2026' ./src/</code>
<code>-A NUM / --after-context=NUM</code>	Prints NUM lines of trailing context immediately following matching lines EXAMPLE: <code>grep -A 3 'Traceback' error.log</code>
<code>-B NUM / --before-context=NUM</code>	Prints NUM lines of leading context immediately preceding matching lines EXAMPLE: <code>grep -B 2 'Failed test' test_output.txt</code>
<code>-C NUM / -NUM / --context=NUM</code>	Prints NUM lines of context both before and after each matching line EXAMPLE: <code>grep -C 2 -i 'database error' app.log</code>
<code>-f FILE / --file=FILE</code>	Obtains search patterns from FILE, one pattern per line; matches any of the patterns EXAMPLE: <code>grep -F -f blocklist.txt requests.log</code>

POSIX Character Classes in Grep

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>[:digit:]</code>	Digits: equivalent to [0-9] EXAMPLE: <code>grep '[:digit:]\+' file.txt</code>
<code>[:alpha:]</code>	Alphabetic letters: equivalent to [a-zA-Z] EXAMPLE: <code>grep '^[:alpha:]\+\$' names.txt</code>
<code>[:alnum:]</code>	Alphanumeric characters: equivalent to [a-zA-Z0-9] EXAMPLE: <code>grep -E '^[:alnum:][_]\+\$' usernames.txt</code>
<code>[:space:]</code>	Whitespace: space, tab, newline, vertical tab, form feed EXAMPLE: <code>grep '[:space:]\+' file.txt</code>
<code>[:blank:]</code>	Horizontal whitespace: space and horizontal tab only EXAMPLE: <code>grep '^[:blank:]*#' config.sh</code>
<code>[:punct:]</code>	Punctuation characters: !"#\$%&'()*+,-./:;<=>?@[\]^_`{ }~ EXAMPLE: <code>grep '[:punct:]' tokens.txt</code>
<code>[:xdigit:]</code>	Hexadecimal digits: equivalent to [0-9a-fA-F] EXAMPLE: <code>grep -E '#[:xdigit:]{6}' styles.css</code>
<code>[:lower:]</code> and <code>[:upper:]</code>	Lowercase letters [a-z] and Uppercase letters [A-Z] EXAMPLE: <code>grep '^[:upper:][:lower:]\+' names.txt</code>